Lesson 1 Python Introduction and Installation

1. Python Introduction

1.1 Object Oriented Introduction

Python is a object oriented programming language. Object Oriented is a method of software development and a programming paradigm. Difference between Object Oriented and Procedure Oriented is as follow.

Software Development Method	Programming Idea
Procedure Oriented	Focus on procedure, analyze the steps to solve problems and implement the steps sequentially with functions.
Object Oriented	Focus on object, decompose things that make up the problem into several objects and describe the behavior of an object in the overall solution.

1.2 Python Introduction

First released in 1991, Python is a cross-platform programming language. And it gets its name from comedy troupe Monty Python preferred by its developer.

Python provides ample API (Application Programming Interface) and tools,

which enables the programmer to write extension modules in C, C++ and Cython. In addition, Python compiler can also be integrated into the program which requires scripting language, so it is frequently used to integrate and package the program written in other languages.

Thanks to its syntax, dynamic typing and nature of interpreted language, Python is the programming language adopted by most platforms to write script and develop application. As the version keeps updating and new functions are added, Python is gradually applied in the development of independent and large-scale project.

Note: Python2.0 is no longer maintained by the official since 2020, so it is recommended to use Python3.0 or above.

1.3 Python Feature

1）Easy to master: Python has small amount of keywords, simple structure and clear syntax definition.

2）Easy to read and maintain: the definition of Python code is distinct and it is easy to maintain the source code.

3）Fast running: base layer, multiple standard libraries and the third-party libraries are written in C, which attributes to fast running.

4）Free and open-source

5）Abundant libraries: Python comes with large standard library which can handle various tasks, including regular expressions, documentation generation, unit testing, threading, databases, web browsers, CGI, FTP, and other system-related operations.

6）Transplantable: as Python is open-source, it can be transplanted to

various platforms, such as Linux and Windows.

2. Python Installation

Note: the following operations are based on Ubuntu18.04. As Ubuntu18.04 comes with Python 3.6.9, users who use this system can skip Python installation.

Before installing Python, install virtual machine for system configuration. For specific instructions, please refers to the file in "**2. Linux Basic Lesson->Lesson 2 Environment Configuration in Windows and Lesson 3 Linux Installation and Source Replacement**".

1) Install Python with Ubuntu official apt tool package.

Note: please strictly distinguish lower case and upper case, and keywords can be complemented by "Tab" key.

---

2) Start virtual machine, and click  , and then click  or press "**Ctrl+Alt+T**" to open command line terminal.

3) Take installing python3.8 for example. Input command "**sudo apt-get install python3.8**" command, and then input the password and press Enter to install.

```
hiwonder@ubuntu:~$ sudo apt-get install python3.8
```

4) If the prompt about whether to continue execution, please input "**Y**" and press Enter. If no error is reported, it means that Python3.8 is installed successfully.

```
Need to get 4,542 kB of archives.
After this operation, 18.4 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

5）Input “**python3.8 -V**” command and press Enter to check whether the version is Python3.8.

```
ubuntu@ubuntu-virtual-machine:~$ python3.8 -V
Python 3.8.0
```

3. pyCharm Installation

3.1 Download pyCharm

1) Input command “sudo apt install snapd snapd-xdg-open” to install snap installation package.

```
hiwonder@hiwonder-virtual-machine:~$ sudo apt install snapd snapd-xdg-open
```

2) Input command “snap refresh” to refresh snap.

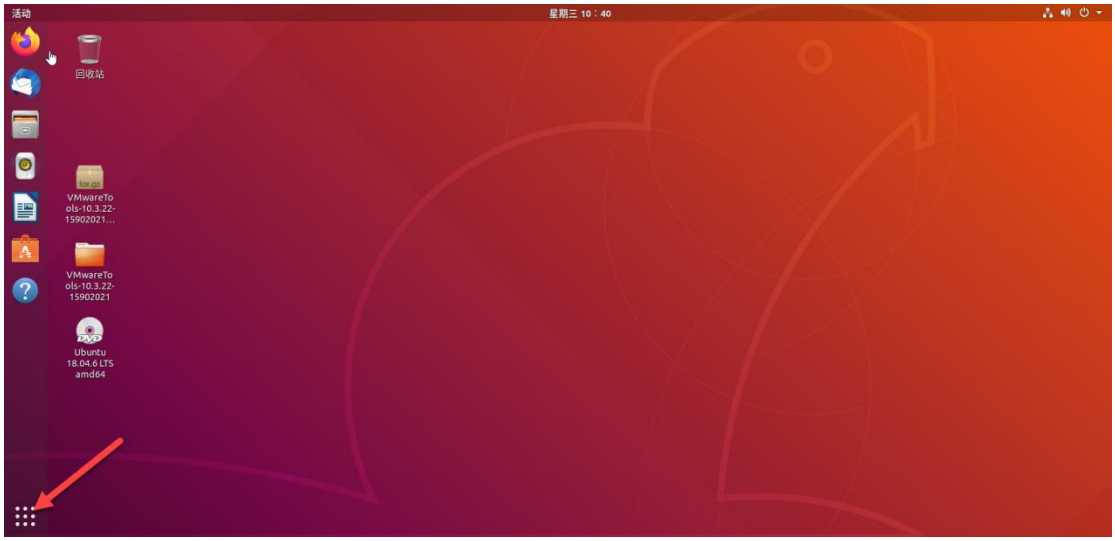
```
hiwonder@hiwonder-virtual-machine:~$ snap refresh
```

3) Input command “**sudo snap install pycharm-community --classic**” to install pyCharm

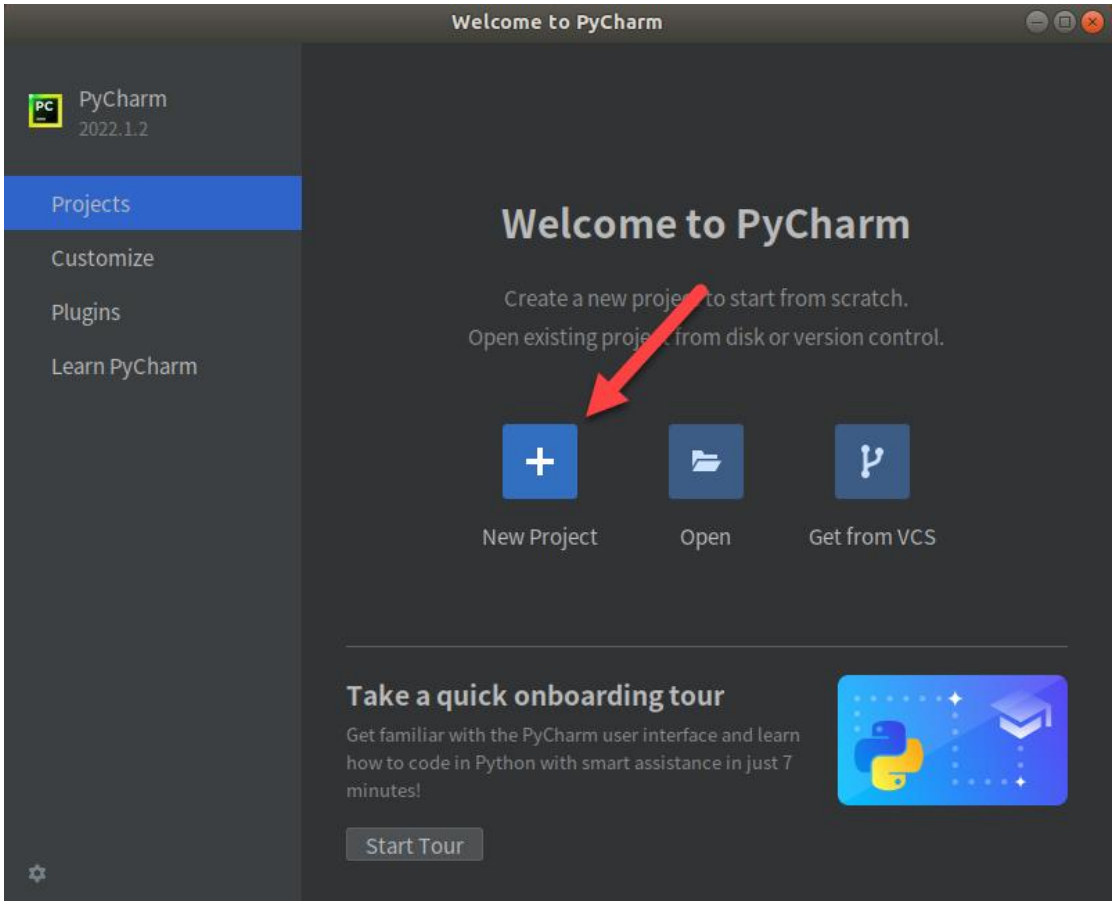
```
hiwonder@hiwonder-virtual-machine:~$ sudo snap install pycharm-community --classic
```

3.2 Open pyCharm

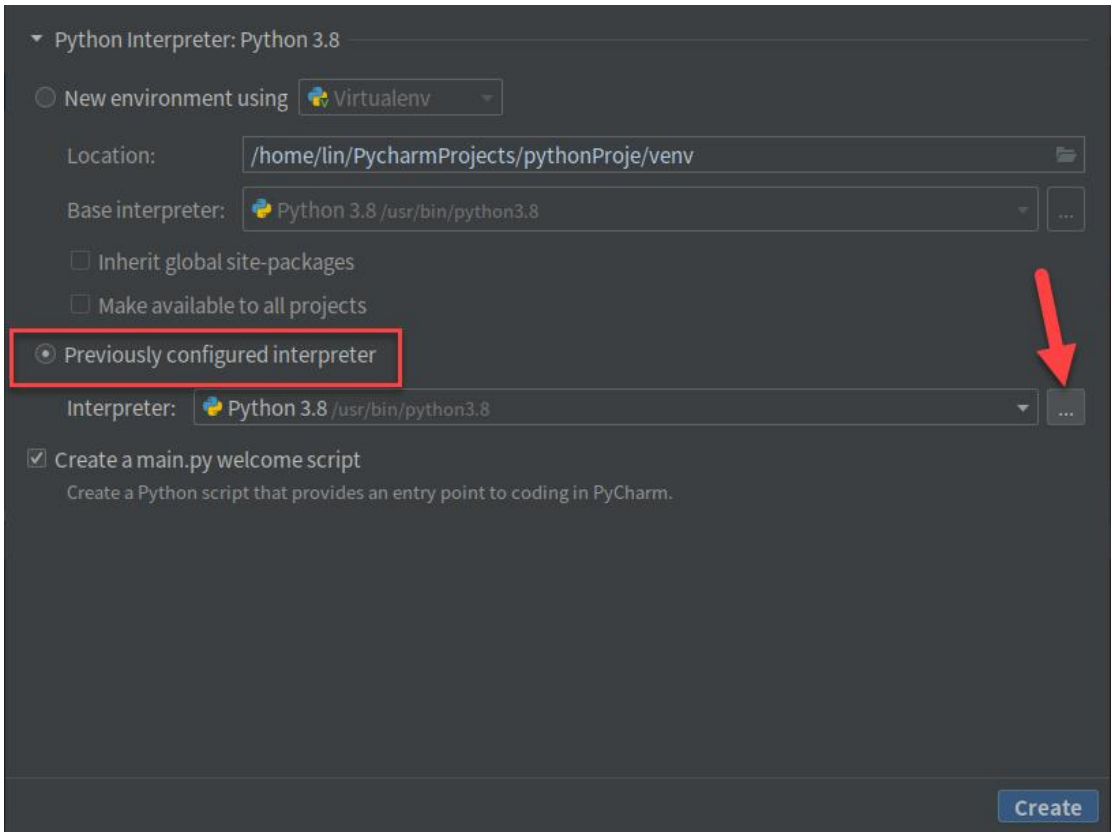
1）Open menu and find pyCharm



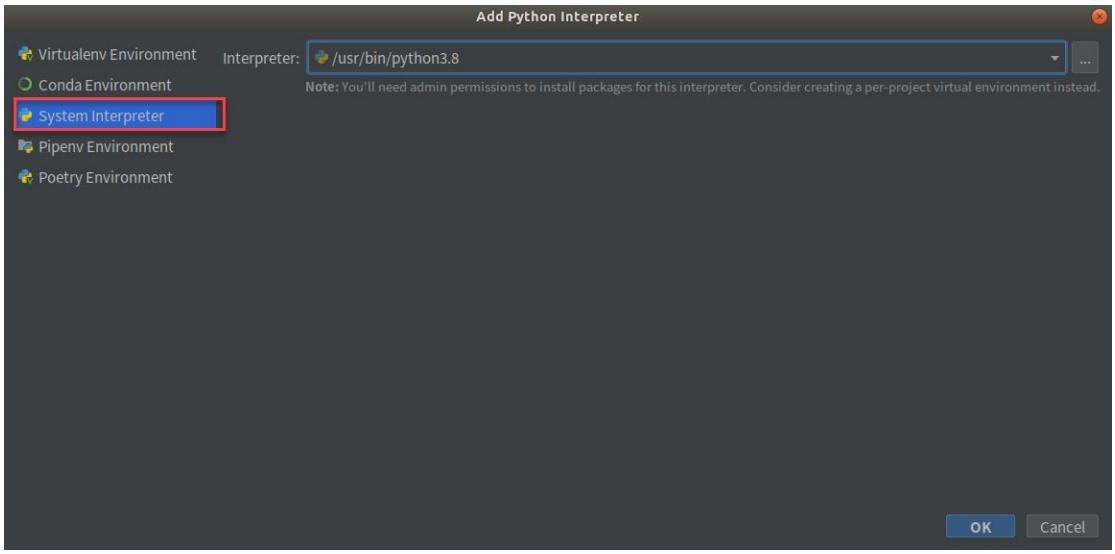
2) Create new pyCharm project and configure. Click “New Project”.



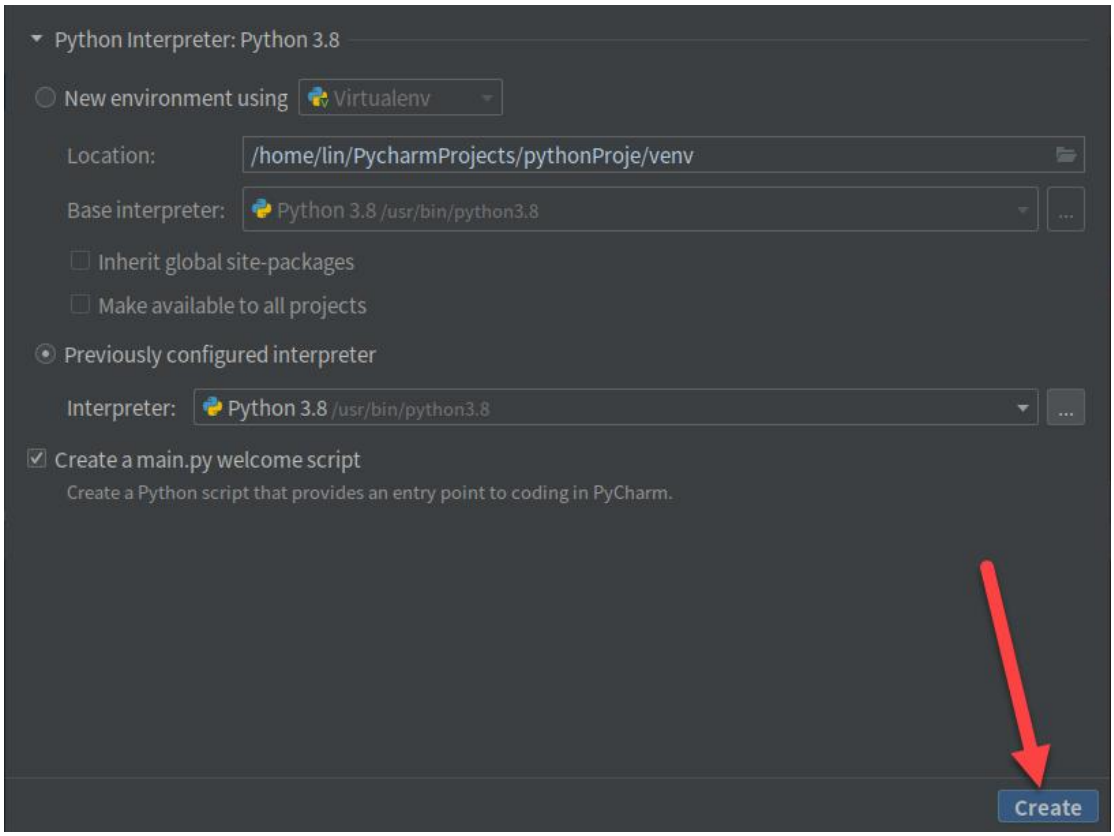
3) Select "**Previously configured interpreter**" and click ...



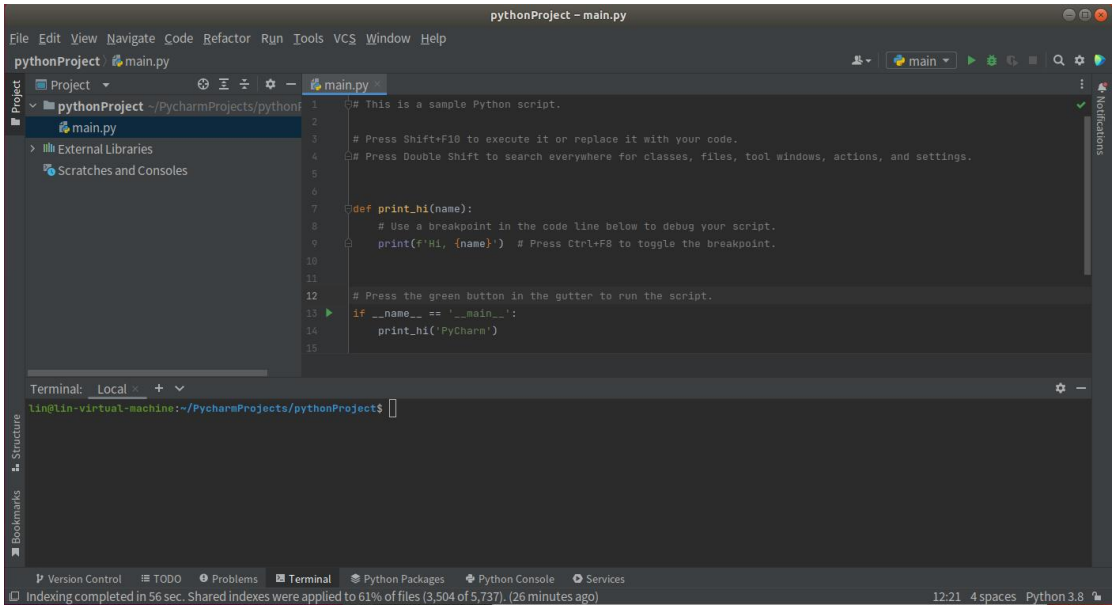
4) Select "**System interpreter**"



5) Click "**create**"

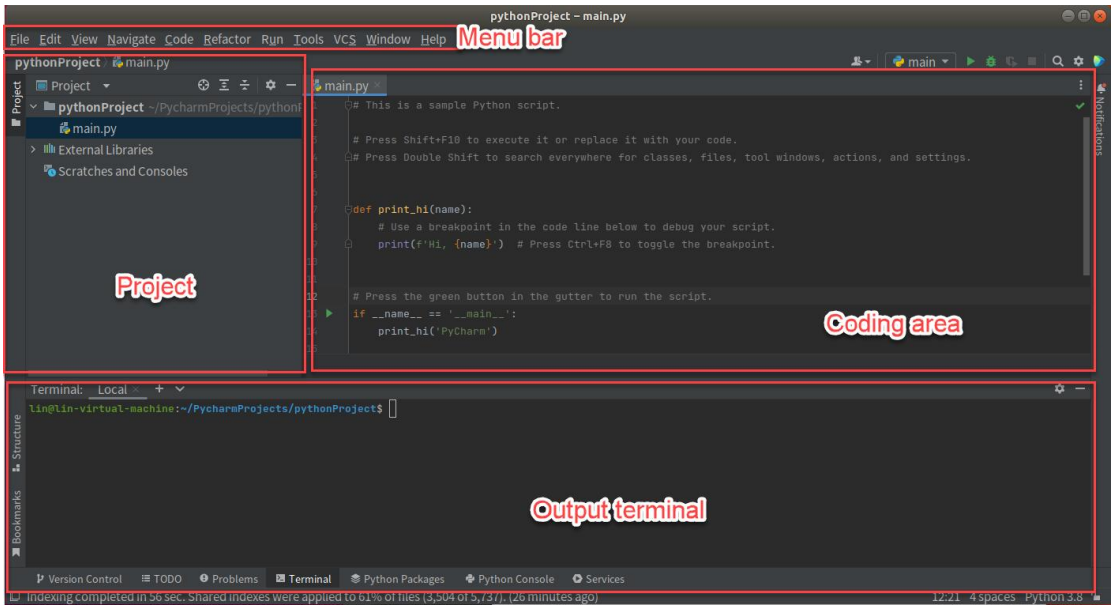


6） Lastly, you will enter this interface.

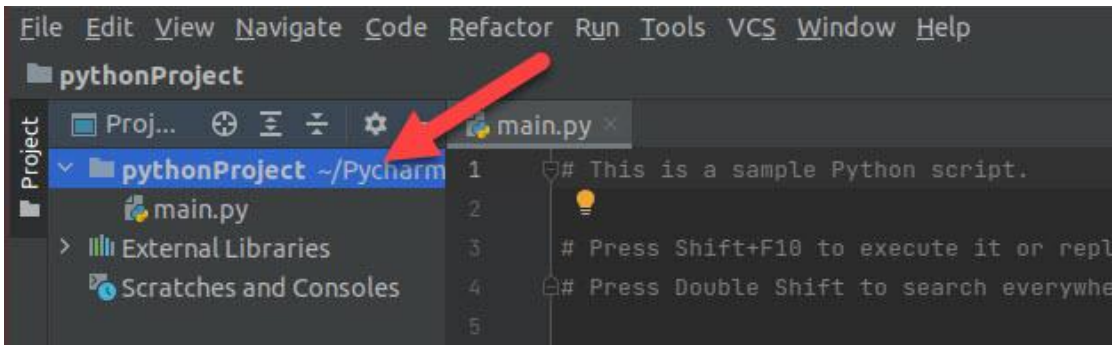


3.3 Usage of pyCharm

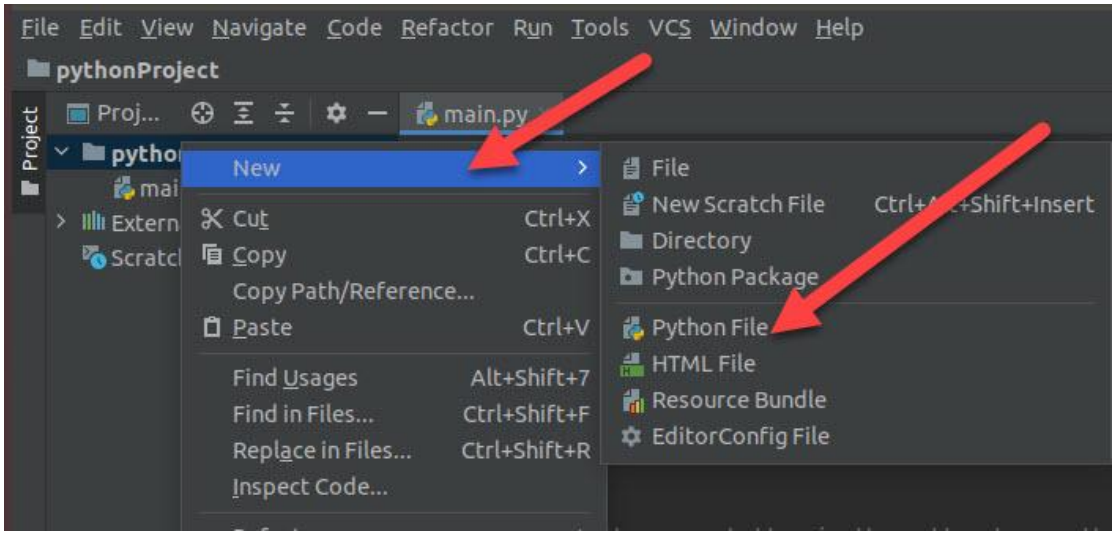
1) The user interface of pyCharm is as follow.



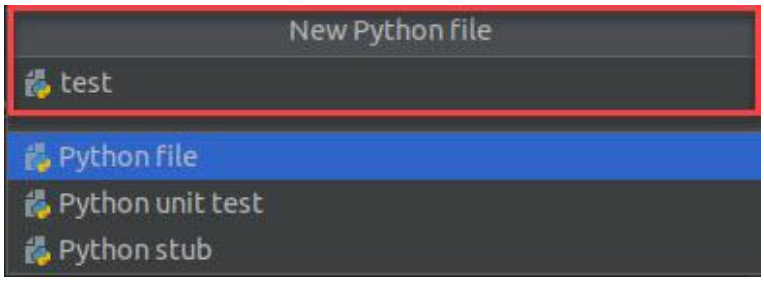
2) Create a .py file. Right click the project folder.



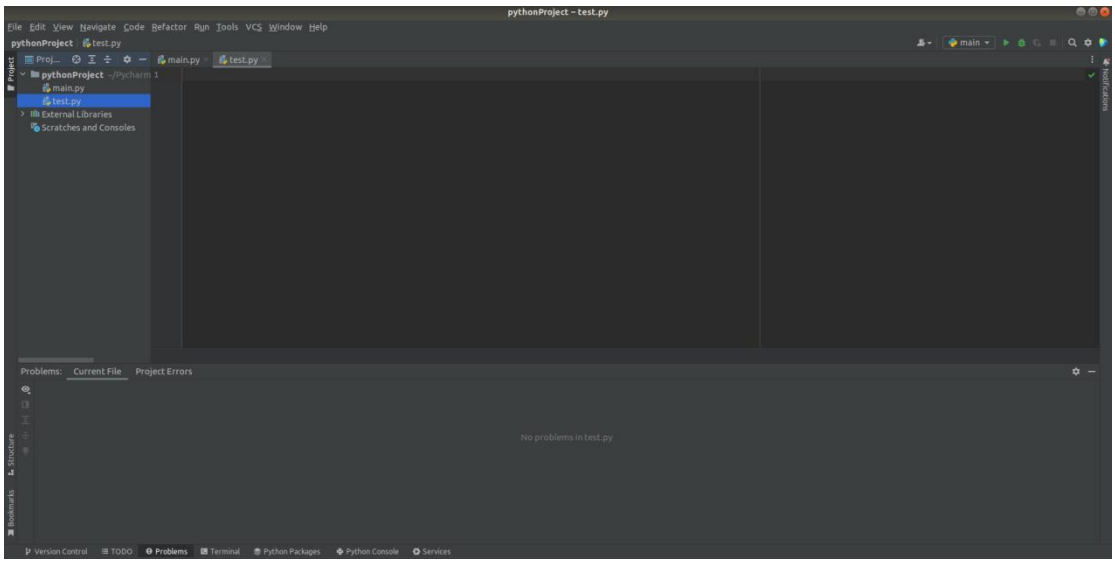
3) Click "**New>Python file**"



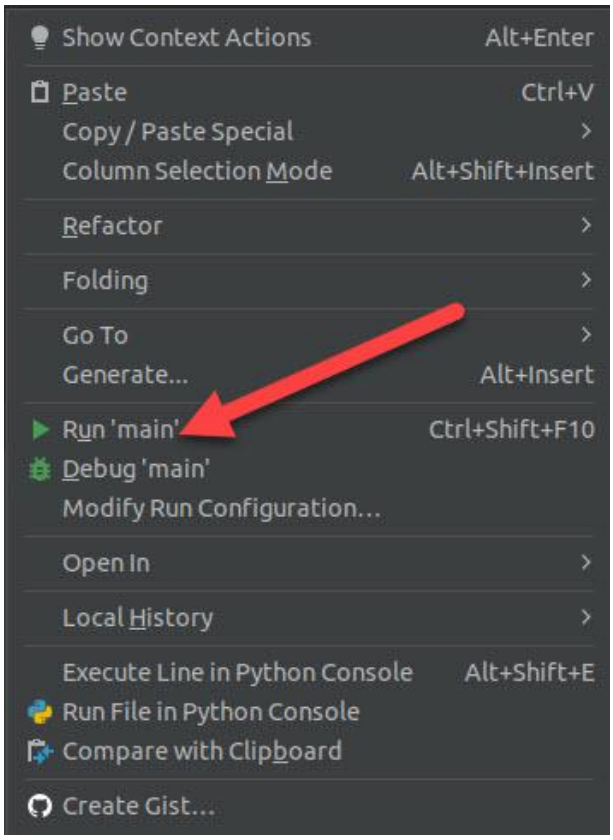
4) Name python file you created.

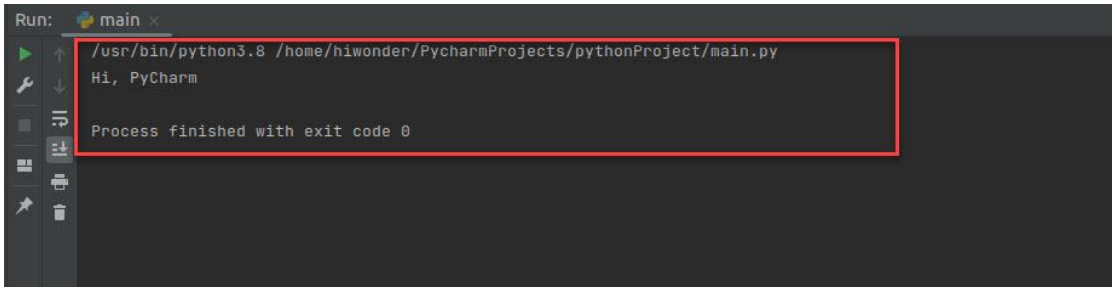


5) The python is created.



6) Right click coding area to select “**run**”. You can check the output result on the output terminal.





To learn more information, please visit pyCharm official website:

https://www.jetbrains.com/zh-cn/pycharm/

Lesson 2 First Program-“Hello World”

String is a collection of multiple characters and enclosed by single or double quotation mark. String includes alphabet, Arabic numeral, Chinese or various symbols.

For example, we can print “**Hello World**” string on the screen.

1. Operation Steps

Note: please strictly distinguish lower case and upper case, and the keywords can be complemented by Tab key.

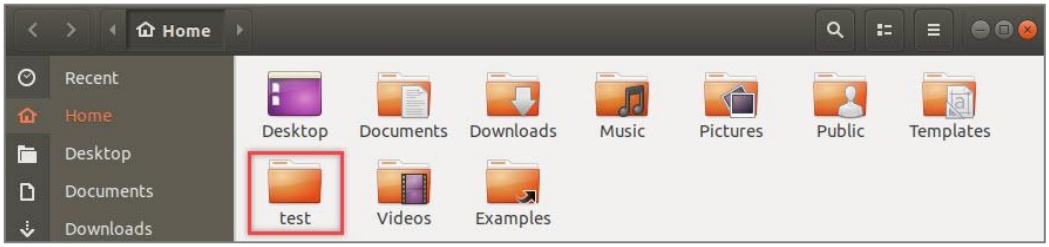
1） Start virtual machine, and click  , and then click  or press “**Ctrl+Alt+T**” to open command line terminal.

2） Input “**sudo apt install vim**” command to install vim editor. During installation, if the prompt about whether to continue execution occurs, just input “**y**” and press Enter.

```
hiwonder@ubuntu:~$ sudo apt install vim
[sudo] password for hiwonder:
```

3） Input “**mkdir test**” command and press Enter to build a folder named “**test**” under the current directory.

```
hiwonder@hiwonder-virtual-machine:~$ mkdir test
```



4） Input command “**cd test/**” and press Enter to enter “**test**” folder.

```
hiwonder@hiwonder-virtual-machine:~$ cd test/
```

5）Input “**touch hello.py**” command and press Enter to create a program file named “**hello**”.

```
hiwonder@hiwonder-virtual-machine:~/test$ touch hello.py
```

6）Input “**vim hello.py**” command and press Enter to open program file.

```
hiwonder@hiwonder-virtual-machine:~/test$ vim hello.py
```

7）Press “**I**” key to enter editing mode and then input “**print("Hello World")**”.

```
print("Hello World")

~
-- 插入 --                                                    2,1
```

8）Press “**Esc**” and input “**:wq**” and press Enter to save and exit the editing.

```
~
:wq
```

9）Input “**python3 hello.py**” command and press Enter to run the program file. Then the string will be printed on the terminal.

```
hiwonder@hiwonder-virtual-machine:~/test$ python3 hello.py
Hello World
```

2. Expansion Content

Besides the string, print() function can be also used to output the result of mathematical expression. Take adding print() function of mathematical expression to the program file for example.

1）Start virtual machine, and click  , and then click  or press “**Ctrl+Alt+T**” to open command line terminal.

2）Input command “**cd test/**” and press **Enter** to enter “**cd test/**” folder.

```
hiwonder@hiwonder-virtual-machine:~$ cd test/
```

3）Input “**vim hello.py**” command, and press Enter to open program file.

```
hiwonder@hiwonder-virtual-machine:~/test$ vim hello.py
```

4）Press “**I**” key to enter the editing mode and input “**print(100+100)**”.

```
print("Hello World")
print(100+100)
```

Note: there is no need to enclose the mathematical expression with double quotation mark.

5）Press “**Esc**” and input “**:wq**” and press **Enter** to save and exit the editing.

```
~
:wq
```

6）Input “**python3 hello.py**” command and press Enter to run the program file. Then the result of the mathematical expression will be printed on the terminal.

```
hiwonder@hiwonder-virtual-machine:~/test$ python3 hello.py
Hello World
200
```

3. Function Explanation

print() function is used to print output in the format below.

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

The first parameter “**objects**” is the output object. When output several objects, they should be separately with “**,**” in between.

The second parameter “**sep**” is used to put string between the output objects, ’ ’ by default.

The third parameter “**end**” is used to add string at the end of output, ’\n’ by default.

The fourth parameter “**file**” is the object with a write function, the default value is “sys.stdout”, that is screen.

The fifth parameter “**flush**” is used to output cache and the default value is “**False**”.

Lesson 3 Python Basic Syntax

This lesson will explain the basic syntax of Python, such as comments, indentation rules, coding standards, etc.

1. Comment

Comments are used to explain Python code. Python support two types of comments, including single line comment and multi line comments.

1) Single Line Comment

Comment starts with “#” and its format is as follow.

```
# comment
```

2) Multi Line Comments

Insert three single quotation marks “'''” or three double quotation marks “"""” at the beginning and the end of the comment to comment multiple lines, and the format is as follow.

```
'''
comment
comment
'''
```

```
"""
comment
comment
"""
```

2. Indentation Rules

Python uses indentation and colon symbol (:) for showing where blocks of code begin and end

In Python, for class definitions, function definitions, flow control statements, exception handling statements, etc., the colon at the end of the line and the indentation of the next line represent the beginning of the next

code block. And when indentation ends, a block of code ends.

Each red frame in the picture below represents one block of code.

```
1  if num>100
2      print("num>100")
3  if num>50 and num<=100:
4      print("50<num<=100")
5  if num<=50:
6      print("num<=50")
```

Note: block of code at the same level should be indented consistently. We can use "**Tab**" key or input 4 spaces to indent.

Note: Python uses 4 spaces as indentation by default. In general, one "Tab" is equal to 4 spaces.

---

3. Coding Standard

Python adopts PEP8 as coding standard. "**PEP**" represents Python Enhancement Proposal, and "**8**" indicates style guide of Python code.

Please strictly follow the coding standard when coding to make the code neater, which will enhance the readability.

1) One "import" is for one module. Please don't import multiple modules for one time.

Recommend	`import os` `import sys`
Not Recommend	`import os,sys`

2) Please don't put semicolon ";" at the end of the line, and don't put two commands at the same line

Recommend	print("Hello World") print("Hello Hiwonder")
Not Recommend	print("Hello World");print("Hello Hiwonder")

3） The length of line should not be greater than 80 characters, and you can separate a command into several lines, and put the command inside "**()**", as the example shown below. It is not recommended to use backslash "\" to connect the lines of contents.

Recommend	a=("Python is a multi-platform programming language," "provide ample API and tools")
Not Recommend	a="Python is a multi-platform programming language, \ provide ample API and tools"

4） When necessary, we can input space to improve the readability of the code.

5） In general, it is recommended to input space around both sides of operators and commas, and between parameters of function to separate.

4. Identifier Naming Standard

Identifier is the name of variable, function, class, module and other objects. In Python, naming identifier should be consistent with the naming rules.

1） Identifier names in Python can contain letters (A~Z, a-z), underscore (_) and number, and the name should always start with a non-numeric

character.

2）Identifier must be different from keywords/ reserved words in Python. For the definition of keywords/ reserved, please move to "**5. keyword/ reserved word**".

3）Identifier cannot contain **space**, **@, % , $** and other special characters.

结合上述三点规则，下表列举了部分命名合法的标识符与不合法的标识符：
Examples of valid and invalid identifier are listed below.

Valid Identifier	Hiwonder mode01 user_age

Invalid Identifier	`4world # Number cannot be the first character`
	`try # try is the reserve word and can not used as identifier`
	`$money # special character cannot contained`

4）Identifier is case sensitive. For example, "num", "Num" and "NUM" are three independent variables.

5）Identifier starting with underscore "_" has special meaning. Please avoid using identifier starting with "_" if not necessary.

Identifier	Meaning	Example
Start with single underscore	Class properties that cannot be accessed directly through "from...import*"	_width
Start with double underscore	Exclusive member of class	__add

Start and end with double underscore	Special identifier	__init__

6）Chinese can be used as identifier in Python. To avoid error, please shun Chinese.

Besides the rules mentioned above, there are corresponding rules in identifier naming under different situation.

1）When used as module name, identifier should be short and composed of lower case letters. And underscore "_" can be used for separation.

2）When used as package name, identifier should be short and composed of lower case letters, but it is not recommended to use full stop ".", such as "com.mr" and "com.mr.book".

3）When used as class name, identifier should start with upper case letters, for example "Book" which defines a book class.

4）When used as class name inside module, identifier can start with "_" and upper case letters, for example "_Book".

5）When used as function name, property name and method name in classes, identifier should be composed of lower case letters and underscore can be used to separate different words.

6）When used as constant name, identifier should consist of upper case letters and different words can be separated by underscore "_".

5. Keyword/ Reserved Word

Reserved word, also called keyword, is a word with special meaning in

Python. And they cannot be used as variable names, function names, class names, module names or any other object names

In Python interactive programming environment, we can check the reserved words according to the steps below.

1） Start virtual machine, and click  , and then click  or press "**Ctrl+Alt+T**" to open command line terminal.

2） Input "**python3**" command and press Enter to enter Python interactive programming environment.

```
hiwonder@hiwonder-virtual-machine:~$ python3
```

3） Input "**import keyword**" command and press Enter to import "**keyword**" module.

```
>>> import keyword
```

4） Input "**keyword.kwlist**" commend and press Enter to view all reserved words in Python.

```
>>> keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del',
'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal',
 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

Note: Reserved words are also case sensitive. For example, "if" is reserved word, while "IF", "iF" and "If" are not.

---

6. Data Type

There are six types of data in Python3, including Number, String, List, Tuple, Dictionary and Set.

And Number, String and Tuple are immutable, while List, Dictionary and

Set are mutable.

Note: In Python, type() function can be used to check the type of variable.

6.1 Number

Number includes three numeric types to represent numbers or value.

1） int: Integers can be binary, octal, decimal and hexadecimal values.

2） float: floats are decimal values generally composed of integers and decimals. Each floating point number occupy 8 bytes, that is 64 bits.

3） complex: complex number is a number with real and imaginary components. Both real and imaginary components belong to floating type.

4） bool: Only has “**True**” and “**False**” values. “**True**” corresponds to “**1**” and “False” corresponds to “**0**”

6.2 String

A string is a collection of multiple characters. Strings in Python are surrounded by either single quotation marks “**'**”, or double quotation marks “**"**” and triple quotation marks “**'''**” or “**"""**”.

Strings with single quotation marks and double quotation marks are equivalent, and return the objects of the same type.

Note: when there is quotation marks inside the strings, we need to escape them through adding backslash “\” in front of the quotation marks to avoid syntax error.

6.3 List

List is a sequence structure in Python, which can store any types of data,

including integer, decimal, string, list, tuple, etc. Its format is as follow.

```
Variable name = [element 1, element 2,..., element n]
```

The number of elements in List is unlimited and there can be different types of elements in one List. For better readability of program, it is recommended to use one type of data in one list.

Each element in the List corresponds to integer index value. The corresponding element values can be obtained though index values so as to change and delete them.

6.4 Tuple

Tuple is another important sequence structure in Python, which is similar to List. And its format is as follow.

```
Variable name = (element 1, element 2,..., element n)
```

Different from List, Tuple is immutable sequence, which means that its elements cannot be changed or deleted.

6.5 Dictionary

A dictionary is an unordered, mutable sequence that is created in the following format.

```
Variable name = (key1:value1, key2:value2,…, keyn:valuen)
```

Dictionary is the only type of mapping in Python, which means that elements corresponds to each other.

The elements of dictionary can be List, Tuple, Dictionary and other types of data, but key value must be the immutable type. In addition, there should be only one key value in the same dictionary variable.

6.6 Set

Set is used to store non-repetitive elements and its format is as follow.

```
variable name = {element 1, element 2,..., element n}
```

Set can only store immutable data type, including integer, float type, string and tuple, but cannot store mutable data type, including List, dictionary and set.

Lesson 4 Python Conditional Statement

1. Conditional Statement Introduction

Conditional statement controls the execution of different blocks of code through judging whether the condition holds and based on the result of condition expression.

1.1 Conditional Expression

Conditional expression consists of operator and operand. Take “**a<4**” for example. “**a**” and “**4**” are operand，and “**<**” is operator.

Judgment conditions can be any element with Boolean properties, including data, variables, and expressions composed of variables and operators. If its Boolean property is "True", the condition holds; if it is "False", the condition doesn’t hold.

The table lists the commonly used operators by conditional expression.

Type	Operator	Mathematical Symbol	Meaning
Arithmetic operator	+	+	add
	-	-	subtraction
	*	×	multiplication
	/	÷	Division, and the result is floating point number
	//		Division, and the result is a

			rounded down integer
	%	%	Modules/ Surplus
	**	^	Exponentiation
Comparison Operators	==	=	Equal
	!=	≠	Not equal
	>	>	Greater than
	<	<	Less than
	>=	≥	Greater than or equal to
	<=	≤	Less than or equal to
Membership Operator	in	∈	Exist/ belong to
	not in	∉	Not exist/ not belong to

Python is able to combine the judgement condition logically through the reserved words “not”, “and” and “or”.

1） not: it represents the “**not**” relationship for an individual condition. If the Boolean property of the “**condition**” is “**True**”, the Boolean property of the “**not condition**” is “**False**”. If Boolean property of the “**condition**” is “**False**”, the Boolean property of the “**not condition**” is “**True**”

2） and: it represents “**and**” relationship between several conditions. Only when the Boolean properties of the conditions connected by “and” are **all “True”**, the Boolean property of the logic expression is “**True**”, otherwise

"**False**".

3） or: it represents "**or**" relationship between several conditions. Only when the Boolean properties of the conditions connected by "or" are **all "False"**, the Boolean property of the logic expression is "**False**", otherwise "**True**".

1.2 Selection Statement

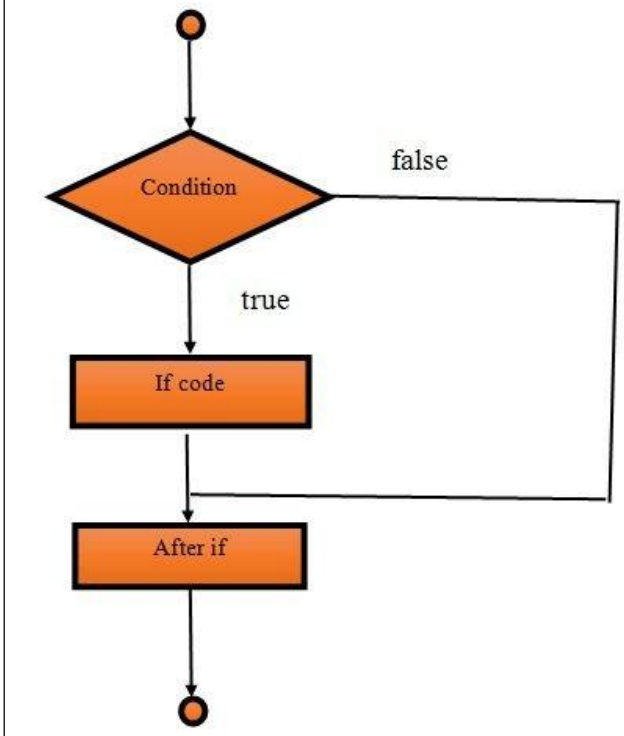
There are three types of selection statements, including single-branch selection structure, double-branch selection structure and multi-branch selection structure.

1) Single-branch selection structure

The syntax and execution process of single-branch selection structure are as follow.

Syntax	Execution Process

```
if condition:
    code block
code block2
```



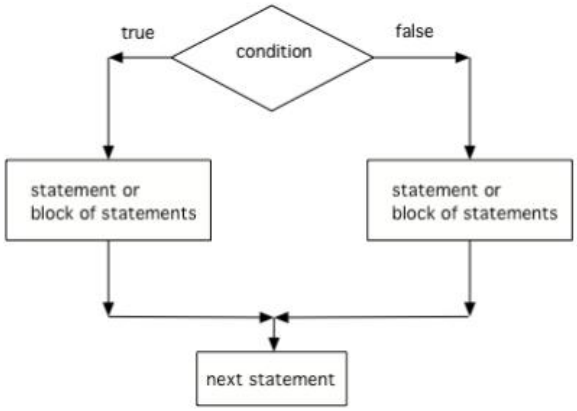
If the condition of “if statement” is true, block of code 1 and 2 will be executed in sequence. Otherwise, skip block of code 1 and directly execute block of code 2.

2) Double-branch selection structure

The syntax and execution process of double-branch selection structure are as follow.

Syntax	Execution Process

```
if condition:
    code block1
else
    code block2
```

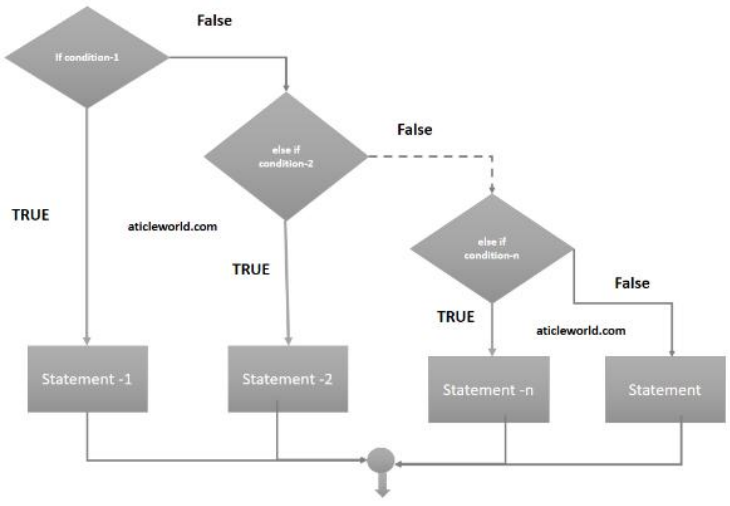


If the condition of “if statement” is true, block of code 1 will be executed. Otherwise, block of code 2 will be executed.

3) Multi-branch selection structure

The syntax and execution process of multi-branch selection structure are as follow.

Syntax	Execution Process

`if condition:`
`  code block1`
`else condition2:`
`  code block2`
`.....`
`elif condition n:`
`  code block n`
`else:`
`  code block n+1`
	

If the condition of “if statement” is true, block of code 1 will be executed.

If the condition of “if statement” is false, conditions of elif statements will be judged in sequence. When the condition is true, the corresponding block of

code will be executed.

If neither conditions of “if statement” nor elif statement are false, block of code n+1 will be executed.

2. Operation Steps

In this routine, the program will calculate BMI value based on the input height and weight, and then perform health assessment.

Before operation, please copy the routine “**conditional_statement.py**” stored in “**3. Python->Python Basic and Advanced Lesson->Lesson 4 Python Conditional Statement-> Routine Code”** to the shared folder.

For the configuration of shared folder, please refer to the file in “**2. Linux Introduction and Usage-> Linux Basic Lesson->Lesson 3 Linux Installation and Source Replacement**”.

Note: the input command should be case sensitive and keywords can be complemented by “Tab” key.

1） Start virtual machine, and click “  ”, and then click “  ” or press “**Ctrl+Alt+T**” to open command line terminal.

2） Input “**cd /mnt/hgfs/Share/**” command and press Enter to enter the shared folder.

```
hiwonder@ubuntu:~$ cd /mnt/hgfs/Share/
```

3） Input “**python3 conditional_statement.py**” command and press Enter to run the routine.

```
hiwonder@ubuntu:/mnt/hgfs/Share$ python3 conditional_statement.py
```

3. Program Outcome

nput height and weight in sequence, and press Enter. Then the terminal will print the corresponding BMI and assessment result.

```
height(m): 1.7
weight(kg): 60
BMI: 20.761245674740486
正常范围，注意保持
```

4. Program Analysis

The routine can be found in "**4. OpenCV Computer Vision Lesson->Image Processing->Lesson 5 Image Processing--Morphological Processing->Routine Code**".

```
1  height = float(input("height(m): "))
2  weight = float(input("weight(kg): "))
3
4  bmi = weight / (height ** 2)    #calculate BMI
5  print("BMI: "+str(bmi))
6
7  if bmi<18.5:
8      print("体重过轻")
9  elif bmi>=18.5 and bmi<24.9:
10     print("正常范围，注意保持")
11 elif bmi>=24.9 and bmi<29.9:
12     print("体重过重")
13 else:
14     print("肥胖")
15
16
```

1) Data input

Call input() function to receive the input data. The hints are inside the parenthesis.

```
1  height = float(input("height(m)：  "))
2  weight = float(input("weight(kg)：  "))
```

2) Data calculation

Process the input data based on BMI calculation formula, and then print the result on the terminal through **print()** function.

```
4  bmi = weight / (height ** 2)    #calculate BMI
5  print("BMI: "+str(bmi))
```

The format of print() function is as follow.

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

The first parameter "**objects**" is the output object. When several objects are output at one time, objects should be separated by comma "**,**".

The second parameter "**sep**" is used to insert the string between the objects. The default value is a space.

The third parameter "**end**" is used to add string at the end of the output. The default value is a line break.

The fourth parameter "**file**" is the object with a write function, the default value is "sys.stdout", that is screen.

The fifth parameter "**flush**" is used to control and output cache. The default value is "**False**".

3) Range Judgement

As there are two or more judgement results, multi-branch structure should be adopted.

```
if bmi<18.5:
    print("体重过轻")
elif bmi>=18.5 and bmi<24.9:
    print("正常范围，注意保持")
elif bmi>=24.9 and bmi<29.9:
    print("体重过重")
else:
    print("肥胖")
```

1）When BMI is less than 18.5, the corresponding BMI value and “underweight” will be printed.

2）When BMI is less than 24.9 and greater than or equal to 18.5, the corresponding BMI value and “normal, please keep it” will be printed.

3）When BMI is less than 29.9 and greater than or equal to 24.9, the corresponding BMI value and “overweight” will be printed.

4）When BMI is greater than 29.9, the corresponding BMI value and “obese” will be printed.

Lesson 5 Python Loop Statement

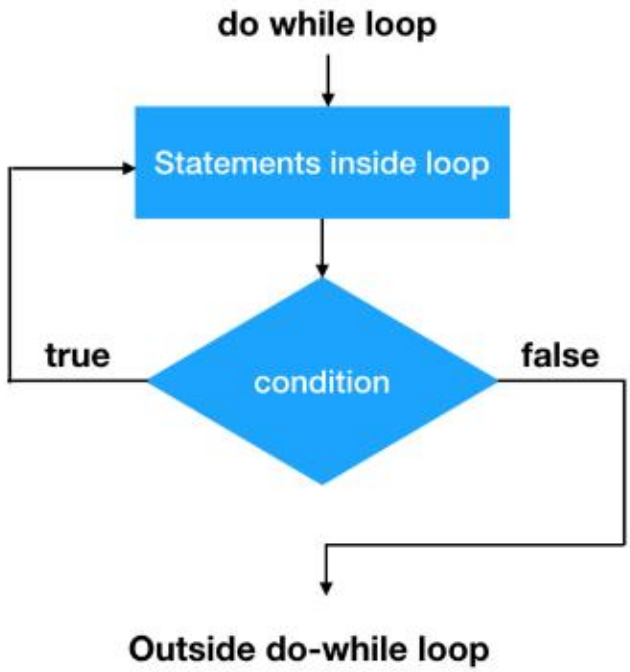
In the program, some steps of the algorithm will be executed repeatedly under certain condition, which is loop statement. In this lesson, Python statement will be explained combining with the related routines.

1. Loop Statement Introduction

Loop statement includes "**while**" and "**for**", which is used to repeat some steps.

1.1 while loop

while 循环的语法格式和执行流程如下：The format and execution process of while loop are as follow.

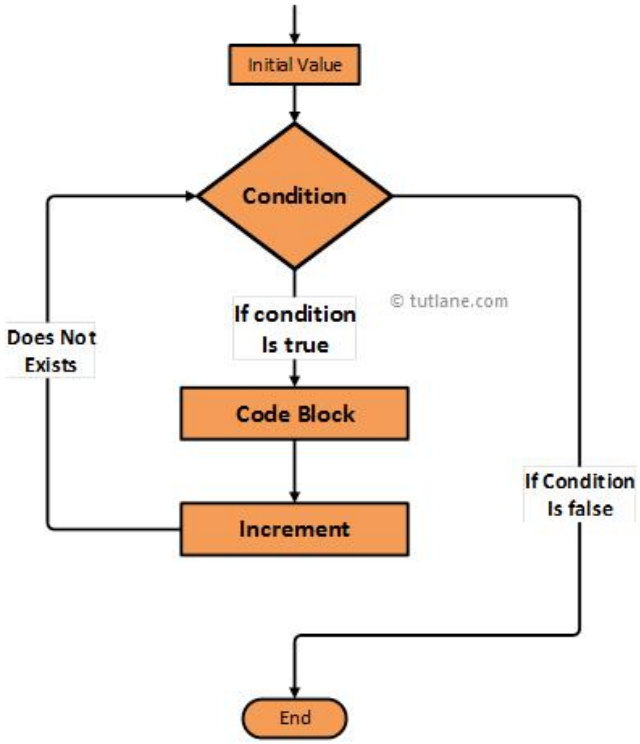
Syntax	Execution Process
`while loop condition:` `    code block`	

When the loop condition is "**True**", the block of code in while loop will be

executed till the loop condition is “**False**”. When the loop condition is always “**True**”, it will fall into “**dead loop**”.

1.2 for loop

The syntax and execution process of for loop are as follow.

Syntax	Execution Process
`for loop variable in target:` `    code block`	

”**Target**” can be string, list, tuple, dictionary and other sequence type, while “**loop variable**” is used to store the elements read from the variable of the sequence type

After entering **for loop**, traverse the elements within target and execute the block of code within **for loop** till the traversal ends.

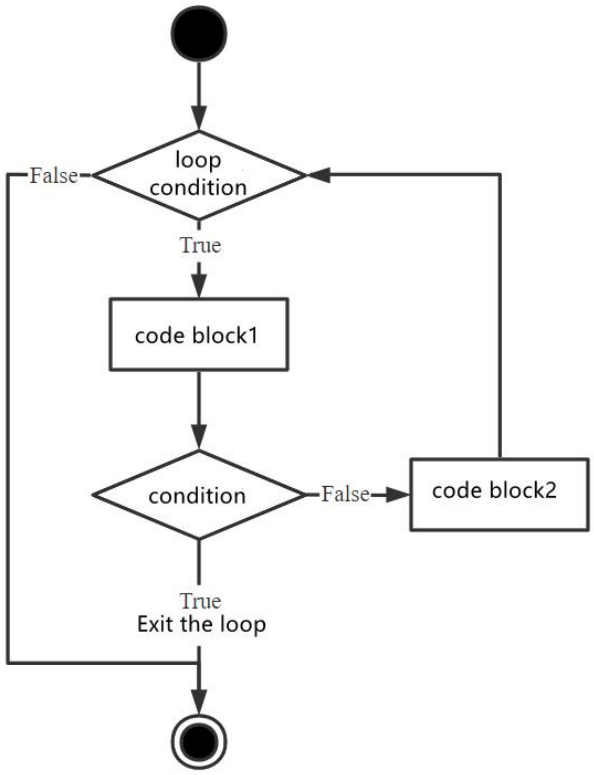
2. Loop Control Statement Introduction

Loop control statement can be used to interrupt the loop, or skip the current loop to execute the next loop. Loop control statement contains "**break**", "**continue**" and "**pass**".

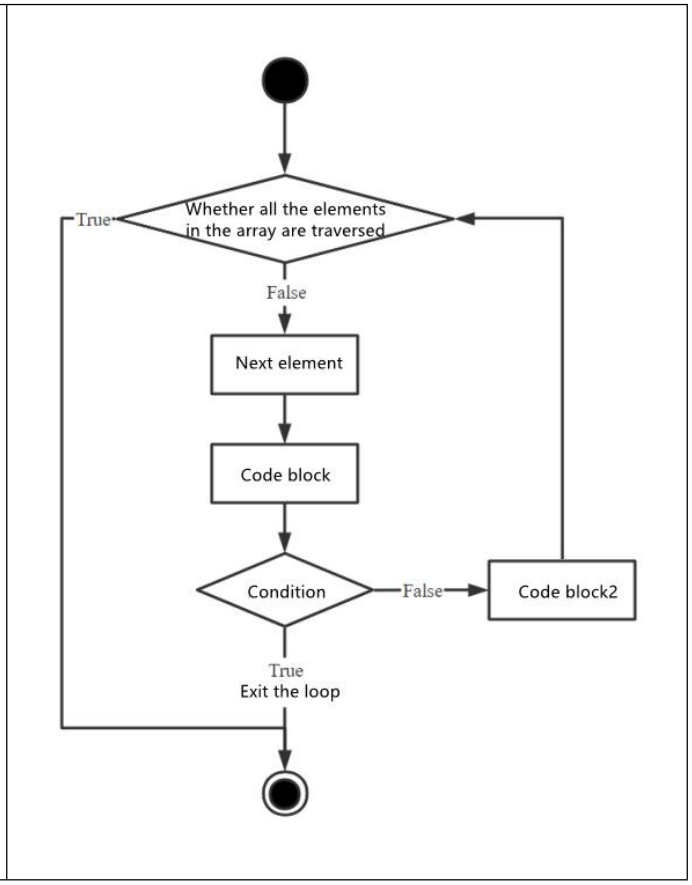
2.1 break Statement

break statement is used to get out of the whole loop. The syntax and execution process are as follow.

Syntax	Execution Process

`while loop condition:`
`  code block1`
`  if condition:`
`  break`
`  code block2`
	

```
for loop variable in target:
    code block1
    if condition:
        break
    code block2
```



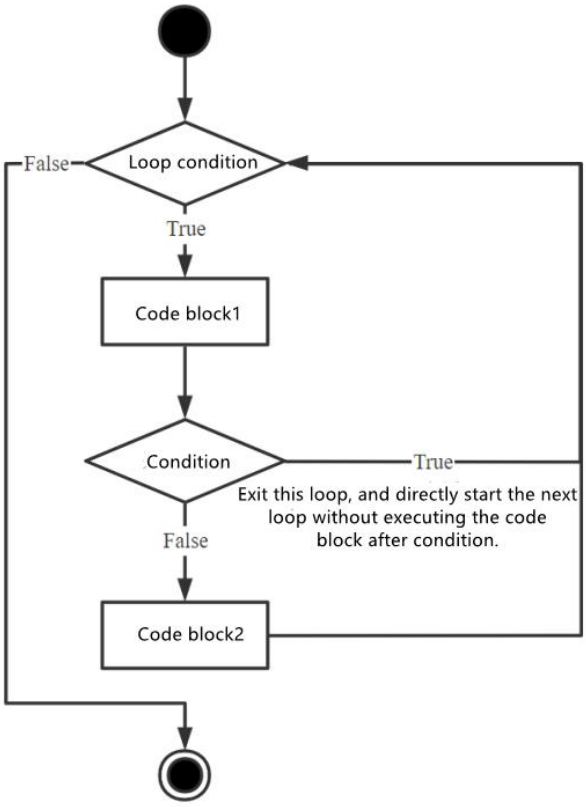
When the judgement condition is executed and is "**True**", the loop will end. If the judgement condition is "False", the block of code 2 will be executed and the loop will continue.

2.2 continue statement

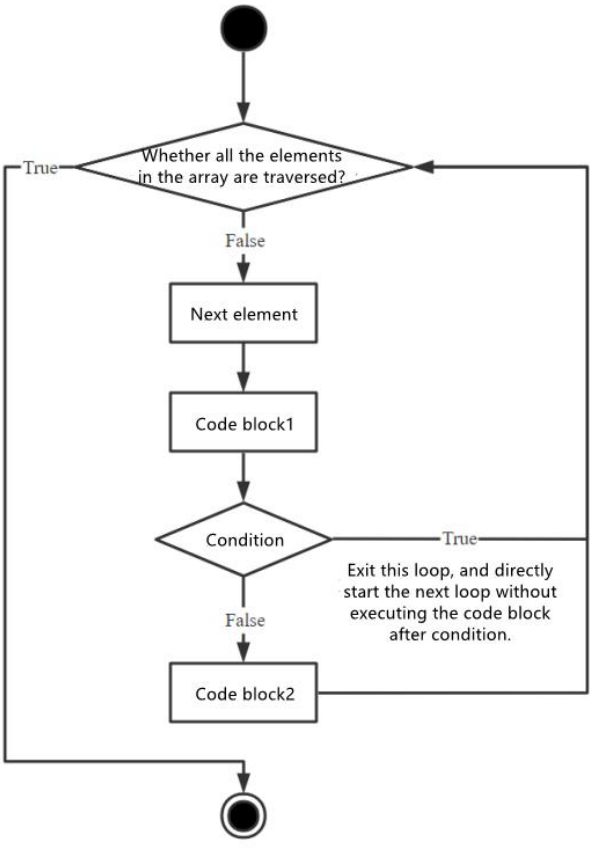
continue statement is used to get out of this round of loop and carry out the next round of loop. The syntax and execution process are as follow.

Syntax	Execution Process

```
while condition:
    code block1(0.001)
    if condition:
        continue
    code block2
```




```
for loop variable in target:
    code block1
    if condition:
        continue
    code block2
```



When the judgement statement is executed and is “**True**”, this round of loop is complete and next round of loop starts. If the judgement condition is “False”, the block of code 2 is executed and the second round of loop is done.

2.3 pass statement

pass statement is null statement and nothing will be executed. It is used to keep the integrity of the program structure.

3. Operation Steps

This routine will calculate the factorial of the integer and print the elements of the string.

Before operation, we need to copy the routine “**loop_statement.py**” stored in “**3. Python->Python Basic and Advanced Lesson-> Lesson 5 Python Loop Statement-> Routine Code**” to the shared folder.

For the configuration of the shared folder, please refer to the file in “**2. Linux Introduction and Usage->Linux Basic Lesson->Lesson 3 Linux Installation and Source Replacement**”.

Note: the input command should be case sensitive, and the keywords can be complemented by “Tab” key.

1） Start virtual machine, and click “  ”, and then click “  ” or press “**Ctrl+Alt+T**” to open command line terminal.

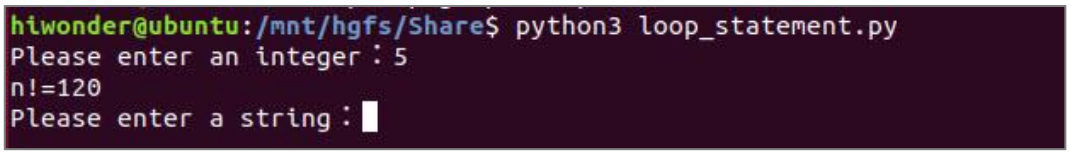
2） Input command “**cd /mnt/hgfs/Share/**” and press Enter to enter the shared folder.

```
hiwonder@ubuntu:~$ cd /mnt/hgfs/Share/
```

3） Input the command “**python3 loop_statement.py**” and press Enter to run the routine.

4. Program Outcome

Input one integer and press Enter, and then the terminal will print its factorial.



Input a string and press Enter, and then the terminal will print the elements of this string.

5. Program Analysis

The used routine “**loop_statement.py**” is stored in “**3. Python->Python Basic and Advanced Lesson-> Lesson 5 Python Loop Statement-> Routine Code**”.

```
n = int(input("请输入一个整数："))
fact = 1
i = 1
while i<= n:
    fact = fact*i
    i = i + 1
print("n!={}".format(fact))

string = input("请输入一个字符串：")
for c in string:
    print(c)
```

5.1 Calculate Factorial of the Value

1） Input data

Call input() function to receive the input data. The hints are inside the parenthesis.

```
n = int(input("请输入一个整数："))
```

2） Create variable

Create two variables for later calculation. “**fact**” is used to store the current factorial.

```
fact = 1
i = 1
```

3） while loop

Use while loop statement to calculate the factorial of the designated integer. During loop, “**i**” value will keep adding up. When this value is greater than the input integer, the loop will end. Then, call print() function to print the result on the terminal.

```
while i<= n:
    fact = fact*i
    i = i + 1
print("n!={}".format(fact))
```

The syntax of print() function is as follow.

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

The first parameter "**objects**" is the output object. When output several objects for one time, objects should be separated by "**,**".

The second parameter "**sep**" is used to insert the string between the objects. The default value is a space.

The third parameter "**end**" is used to add string at the end of the output. The default value is a line break.

The fourth parameter "**file**" is the object with a write function, the default value is "**sys.stdout**", that is screen.

The fifth parameter "**flush**" is used to control and output cache. The default value is "**False**".

5.2 Print String Elements

1) Input data

Call input() function to receive the input data. The hints are inside the parenthesis.

```
9  string = input("请输入一个字符串：")
```

2) For loop

Use for loop to obtain the elements of the input string. Then call print() function in the loop structure to print the input string separately.

```
10 for c in string:
11     print(c)
```

Lesson 6 Python Function and Module

n this lesson, functions and modules of Python will be explained combining the related routine.

1. Function Introduction

In Python, function is divided into built-in function and user-defined function.

Built-in function that readily comes with Python can be directly called without importing any function libraries. The common built-in functions include print(), input(), range(), etc.

User-defined function will define a piece of regular and reusable code as function to elevate the reuse rate and maintenance of the codes.

1.1 Function Definition

In Python, function generally is composed of function name, parameter list and function body consisting of series of statements. And the format is as follow.

```
def function (parameter list):
    function body
```

1）A block of code starts with keyword “**def**”, and function identifier name and parameter list follows.

2）For the convenience of future maintenance, it is better to reflect the function of the **function** on function identifier name.

3）Parameter list is used to set the parameters that can be received by the function, and the parameters can be separated by “**,**”.

4）Any passed in parameters and variables should be inserted inside the

parenthesis.

5）Functions should start with colon "**:**" and are indented strictly.

6）The first line of statement in function can be docstring, i.e. function description.

7）Functions have returned value which is "**None**" by default.

Note: when building function, the parenthesis behind the function name cannot be omitted though there is no parameter in the function.

1.2 Parameter and Argument

When defining function, we should adopt the parameter, while when calling the function, we adopt argument. They both work to pass the data.

1）parameter

Parameters are not actual variables, so they are also called virtual variable. Parameters are adopted when defining function name and function. They are used to receive the parameters passed in when the function is called, and their scope is generally limited to the inside of the function body.

2）argument

Arguments are the parameters passed to the function when calling the function. They can be constants, variables, expressions and functions, and their scope is based on the actual setting.

Regardless of the types, all arguments must have certain values when calling the function so that these values can be sent to parameter.

Take the code below for example. "**width**" and "**height**" are parameters, while "**w**" and "**h**" defined outside the function body are arguments.

```
# area()
def area(width, height):
    return width * height

# call area function
w = 4
h = 9
print("with=", w, "height=", h, "area=", area(w, h))
```

```
with= 4 height= 9 area= 36
```

After parameter “**w**” and “**h**” are passed into the function body, “**width**” and “**height**” parameters are assigned to the corresponding values.

1.3 Return Value

After the function is executed, system will feed back some value to external caller, and these values are considered as return values of functions.

In Python, when function runs to return statement, it means that the function completes running and designated values will be returned. If there is no return statement inside the function, function will return “**None**” by default.

Take the code below for example. After **add()** function is called, this function will return the result of “**x+y**”, and assign variable “**result**” to this value.

```
#function definition
def add(x, y):
    print('x+y=', x + y)
    return x + y

#function calling
result = add(y=1, x=2)
print(result)
```

```
x+y= 3
3
```

Pay attention, in Python, there can be multiple return values in one function as the picture below shown.

```
# function definition
def calculate(x, y):
    print('x+y=', x + y)
    print('x*y=', x * y)
    return x + y, x * y

# function calling
a, b = calculate(y=1, x=2)
print(a, b)
```

```
x+y= 3
x*y= 2
3 2
```

There are two return values inside the **calculate()** function, including "**x+y**" and "**x*y**". After this function is called, "**a**" and "**b**" variables will be assigned to these two return values.

2. Function Passing

There are two types of function parameters, including mutable and immutable, whose calling results are different.

2.1 Mutable Parameter

The calling of mutable parameter is similar to pass-by-reference in C++. If the mutable parameter is passed, such as list and dictionary, modification of the passed in parameters inside the function will affect the external variables.

For example, after the passed in list "**list_01**" is changed inside **change_int()** function, the external variable will also be changed.

```
def change_int(list_01):
    list_01.append([1, 2, 3])  # change the passed-in list
    print("函数内修改后的变量：", list_01)

list_01 = [10, 20, 30]
change_int(list_01)
print("函数外变量的值：", list_01)
```

```
函数内修改后的变量： [10, 20, 30, [1, 2, 3]]
函数外变量的值： [10, 20, 30, [1, 2, 3]]
```

2.2 Immutable Parameter

Calling immutable parameters is similar to C++ pass-by-value. If immutable parameters are called, such as integer, string and tuple, the modification of the passed in parameter inside the function will not affect the external variable.

Take code below for example. The variable "**b**" points to the int object "**2**". When passed to the **unchange_int()** function, the variable "**b**" is copied by value, that is, the variable "**a**" and the variable "**b**" both point to the same int object "**2**" .

However, when execute "**a = 10**", variable "**a**" points to newly generated int object "**10**". Therefore, the external variable doesn't change, and the printed value is "**2**".

```
def unchange_int(a):
    a = 10

b = 2
unchange_int(b)
print(b)
```

```
2
```

3.Parameter Type

3.1 Positional Parameter

When calling function, each argument is associated with the corresponding parameter in positional order, and this association is called a positional parameter.

Take the code below for example. When calling **describe_student()** function, name and age parameters should be offered in sequence. "**Jack**" and "**18**" are respectively stored in "**person_name**" and "**student_age**".

```
def describe_student(person_name, student_age):
    print("My name is ", person_name)
    print(person_name + " is " + student_age + " years old")

describe_student('Jack', '18')
```

```
My name is  Jack
Jack is 18 years old
```

3.2 Default Parameter

When defining functions, we can specify the default value of each parameter. When calling function, if arguments are provided to parameters, adopt the designated argument. Otherwise, the default value of parameter should be adopted.

For example, set the default value of "**student_age**" parameter as "**18**". When argument is offered to "**student_age**" parameter during calling describe_student() function, adopt the designated argument. And the designated argument of this example is "**20**".

```
def describe_student(person_name, student_age='18'):
    print("My name is ", person_name)
    print(person_name + " is " + student_age + " years old")

describe_student('Jack', '20')
describe_student('Jack')
```

```
My name is  Jack
Jack is 20 years old
My name is  Jack
Jack is 18 years old
```

3.3 Variable-length Parameter

In Python, function can also be defined as variable-length parameter

which is also called mutable parameter. By adding "*****" in front of the identifier, the corresponding parameter can be defined as variable-length parameter.

Take the codes below for example. After parameter "**number**" is defined as variable-length parameter, the called **calculate()** function can be directly used even though the passed in parameters are neither list nor tuple.

```
def calculate(*numbers):
    sum = 0
    for n in numbers:
        sum = sum + n
    return sum

print(calculate(1, 2, 3, 4))
print(calculate())
```

```
10
0
```

3.4 Keyword Parameter

Keyword parameter will be passed by "**parameter name-value**" pair. In this way, when designating the argument of function, the position of argument and parameter can be different and we just need to ensure the parameter name is correct.

Check the code below. When passing the parameter, the function can output normally though the parameter order is adjusted.

```
def describe_student(person_name, student_age):
    print(person_name + " is " + student_age + " years old")

describe_student(person_name='Jack', student_age='18')
describe_student(student_age='20', person_name='Mike')
```

```
Jack is 18 years old
Mike is 20 years old
```

3.5 Named Keyword Argument

Named keyword arguments can be used when it is necessary to restrict parameters to be passed only by keyword. In user-defined function, parameters are separated by "*" and the parameters following "*" are named keyword parameter.

Take the codes below for example. "**live_city" following** "*" is named keyword parameter. Therefore, the parameters must be passed by keyword, otherwise the program will throw error.

```
def describe_student(person_name, student_age, *, live_city):
    print("Name：  " + student_age)
    print("Age：  " + student_age)
    print("City：  " + live_city)

describe_student('Jack', '18', live_city='Guangzhou')
```

```
Name: 18
Age: 18
City: Guangzhou
```

Note: In Python, the parameters should be defined in order that is positional parameter, default parameter, mutable parameter, named keyword parameter and keyword parameter.

4. Module Introduction

In general, module is a file suffixed by "**.py**". In addition, there is other file type of module, such as "**.pyo**", "**.pyc**", "**.pyd**", "**.so**" and "**.dll**". If you are Python novice, you can skip these types.

Package the code that can realize specific function as an independent module, which is convenient for other programs and scripts to be imported and used, and avoid the function name from overlapping the variable name.

Modules comes from three channels.

1）Python built-in modules (standard library)

2）the third-party module

3）User-defined module

When you need to call the function of a module, import this module through import statement before calling the function. There are two ways to import the module. (content inside the square parenthesis can be omitted)

1）import **module name** [as alias]

All members of the designated module will be imported when we use the import statement in this syntax. When you need to use the members of the module, this module name should be used as the prefix of the member name.

2）from **module name** import **member name**[as alias]

When the import statement in this syntax is adopted, only the designated member of the module will be imported but not all members. At the same time, when this member is used in the program, there is no need to add any prefix to the member name.

Note: “from module name import *” can also be used to import all the members of the designated members, but it is not recommended to use this.
